CSE 4820 – Arduino Microcontroller Concepts

Chapter 5: Simple Digital and Analog Input

Q1. State the logic levels obtained from a push-button connected to Arduino digital pin 7 using the internal pull-up resistor. Also identify the pin-configuration statement required for this interface.

Solution: With INPUT_PULLUP, the input reads HIGH when the switch is open and LOW when the switch is pressed because the pressed switch connects the pin to ground. The required statement is `pinMode(7, INPUT_PULLUP);`.

Q2. Identify the cause of an Arduino switch input that changes randomly between HIGH and LOW when the switch is open. Select a suitable pull-up or pull-down arrangement and state how it establishes a stable logic level.

Solution: The cause is a floating input. A pull-up or pull-down resistor fixes the input at a definite HIGH or LOW state. INPUT_PULLUP may be used to establish a stable HIGH state when the switch is open.

Q3. Consider a mechanical push-button that shows multiple rapid HIGH/LOW transitions during one press. Determine the purpose of debouncing and calculate the press duration if the detected press and release times are 1.25 s and 2.87 s.

Solution: The rapid transitions are switch bounce and may produce multiple detections of one press; debouncing removes these false transitions. Press duration = 2.87 − 1.25 = 1.62 s.

Q4. For a 4 × 3 matrix keypad interfaced to Arduino, determine the number of row/column signal lines required and outline how the active row-column combination identifies the pressed key.

Solution: A 4 × 3 keypad requires 4 + 3 = 7 signal lines. Arduino scans the row and column lines; the detected row-column intersection identifies the pressed key.

Q5. Apply the ADC relationship for a 10-bit Arduino input with a 5 V reference to obtain the digital reading for 2.5 V. Also determine the approximate voltage represented by one ADC count.

Solution: ADC reading = (2.5/5) × 1023 ≈ 512. One ADC count represents approximately 5/1023 = 4.89 mV.

Q6. For an `analogRead( ) value of 614 obtained with a 5 V reference, determine the corresponding input voltage and obtain the approximate ADC reading that would represent 4 V.

Solution: Vin = 614 × 5/1023 ≈ 3.00 V. For 4 V, ADC ≈ (4/5) × 1023 ≈ 818.

Q7. Use the Arduino `map()` concept to convert an input range of 0–1023 to an output range of 0–255. Determine the mapped value for an input of 512 and indicate one application of this conversion.

Solution: The mapped value is approximately (512/1023) × 255 ≈ 128. One application is converting an analog reading into a PWM range for LED brightness control.

Q8. A system has more analog sensors than available Arduino analog input pins. Select an appropriate method using a 4051 multiplexer and indicate how its selector lines are used to access the individual sensor channels.

Solution: A 4051 analog multiplexer can connect several sensor channels to a common Arduino analog input. Its selector lines determine which channel is connected at a given time, allowing the sensors to be read sequentially.

Q9. For a 0–5 V, 10-bit ADC, determine the approximate digital thresholds corresponding to 2 V and 4 V for an application that must distinguish three input regions.

Solution: For 2 V, ADC ≈ (2/5) × 1023 ≈ 409. For 4 V, ADC ≈ (4/5) × 1023 ≈ 818. The three regions are below 409, from about 409 to 818, and above 818.

Q10. Compare the voltage calculated from an `analogRead()` value of 512 when the reference is assumed to be 5 V and when it is 4.096 V. Comment on the effect of the reference-voltage assumption on measurement accuracy.

Solution: For 5 V, Vin ≈ 512 × 5/1023 ≈ 2.50 V. For 4.096 V, Vin ≈ 512 × 4.096/1023 ≈ 2.05 V. Thus the reference voltage directly affects the calculated voltage; using an incorrect reference introduces error.

Q11. Determine the output voltage of a divider used to interface a 12 V source to an Arduino analog input when R1 = 30 kΩ and R2 = 10 kΩ. State why the divider is required in this application.

Solution: Vout = Vin × R2/(R1+R2) = 12 × 10/(30+10) = 3 V. The divider scales the higher source voltage to a safe level for the Arduino analog input.

Q12. Develop a short Arduino program that reads a push-button on digital pin 7 using the internal pull-up resistor and turns an LED connected to pin 13 ON whenever the button is pressed and OFF otherwise.

Solution:

```
pinMode(7,INPUT_PULLUP);
pinMode(13,OUTPUT);
int state = digitalRead(7);

if (state == LOW)

 digitalWrite(13, HIGH);

else

digitalWrite(13, LOW);
```

The LED is ON when the button is pressed because INPUT_PULLUP makes the pressed state LOW.

Q13. Perform a dry run of the following Arduino code and determine the value printed on the Serial Monitor for the variable `value` when the potentiometer is at the stated position.

```
int sensorValue; int value;

void loop()  { sensorValue = analogRead(A0);

value = map(sensorValue, 0, 1023, 0, 100);
```

Serial.println(value);

}

if analogRead(A0)

returns 512, predict the output.

Solution: For sensorValue = 512, value = map(512, 0, 1023, 0, 100). This gives approximately 50.

Therefore, the Serial Monitor prints approximately 50.

Q14. Analyse an Arduino system in which an LED must indicate three ranges of an analog signal: below 2 V, 2–4 V, and above 4 V. Establish the corresponding 10-bit ADC boundaries for a 5 V reference and state the three decision regions.

Solution: 2 V corresponds to about 409 and 4 V to about 818. Hence the three regions are ADC < 409; 409 ≤ ADC ≤ 818; and ADC > 818. Each region can be assigned a different LED indication.

Q15. Trace the signal-processing path of an Arduino application in which a potentiometer controls an LED response. Identify the roles of `analogRead()`, range mapping, and output control in converting the input voltage into the final LED response.

Solution: The potentiometer produces a variable voltage → `analogRead()` converts it to an ADC value → the value is mapped to the required output range → the mapped value controls the LED, for example through PWM brightness.